

EXPERIMENT 6

AIM:

Create a stored procedure transfer_employee(emp_id, new_dept_id) with validation and error handling. Implement triggers for salary validation and audit logging. Test edge cases.

SQL QUERIES

1. Create Audit Table

First, create a table to store salary changes.

```
CREATE TABLE Salary_Audit (
    Audit_ID INT AUTO_INCREMENT PRIMARY KEY,
    Emp_ID INT NOT NULL,
    Old_Salary DECIMAL(10,2),
    New_Salary DECIMAL(10,2),
    Changed_At TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    Action_Type VARCHAR(20)
);
```

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		2	0	0

2. Trigger for Salary Validation

Create a BEFORE INSERT trigger:

```
CREATE TRIGGER validate_salary_insert
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
    IF NEW.Salary IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be NULL';

    ELSEIF NEW.Salary <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be greater than zero';

    ELSEIF NEW.Salary < 10000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be less than 10000';

    ELSEIF NEW.Salary > 1000000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot exceed 1000000';
    END IF;
END
```

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		2	0	0

SHOW TRIGGERS;

Trigger	Event	Table	Statement
validate_salary_insert	INSERT	employee	BEGIN IF NEW.Salary IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be NULL'; ELSEIF NEW.Salary <= 0 THEN SIG
validate_salary_update	UPDATE	employee	BEGIN IF NEW.Salary IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be NULL'; ELSEIF NEW.Salary <= 0 THEN SIG

9. Salary Audit Trigger

This trigger automatically records every successful salary change.

```
CREATE TRIGGER salary_audit
AFTER UPDATE ON Employee
FOR EACH ROW
```

```

BEGIN
  IF OLD.Salary <> NEW.Salary THEN
    INSERT INTO Salary_Audit
    (
      Emp_ID,
      Old_Salary,
      New_Salary,
      Action_Type
    )
    VALUES
    (
      NEW.Emp_ID,
      OLD.Salary,
      NEW.Salary,
      'SALARY UPDATE'
    );
  END IF;
END

```

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		2	0	0

SHOW TRIGGERS;

Trigger	Event	Table	Statement
validate_salary_insert	INSERT	employee	BEGIN IF NEW.Salary IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be NULL'; ELSEIF NEW.Salary <= 0 THEN SIG
validate_salary_update	UPDATE	employee	BEGIN IF NEW.Salary IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be NULL'; ELSEIF NEW.Salary <= 0 THEN SIG
salary_audit	UPDATE	employee	BEGIN IF OLD.Salary <> NEW.Salary THEN INSERT INTO Salary_Audit (Emp_ID, Old_Salary, New_Salary, Action_Type) VALUES (NEW.Emp_ID, OLI

10. Create Stored Procedure

The procedure transfer_employee() transfers an employee to another department.

CREATE PROCEDURE transfer_employee(

```

  IN p_emp_id INT,
  IN p_new_dept_id INT
)
```

BEGIN

```

  DECLARE v_emp_count INT DEFAULT 0;
  DECLARE v_dept_count INT DEFAULT 0;
  DECLARE v_current_dept INT DEFAULT 0;

```

DECLARE EXIT HANDLER FOR SQLEXCEPTION

BEGIN

```

  ROLLBACK;
  RESIGNAL;

```

END;

START TRANSACTION;

```

SELECT COUNT(*)
  INTO v_emp_count
  FROM Employee
 WHERE Emp_ID = p_emp_id;

```

```

IF v_emp_count = 0 THEN
  SIGNAL SQLSTATE '45000'
  SET MESSAGE_TEXT = 'Employee does not exist';
END IF;

```

```

SELECT COUNT(*)

```

```

INTO v_dept_count
FROM Department
WHERE Dept_ID = p_new_dept_id;

```

```

IF v_dept_count = 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Department does not exist';
END IF;

```

```

SELECT Dept_ID
INTO v_current_dept
FROM Employee
WHERE Emp_ID = p_emp_id;

```

```

IF v_current_dept = p_new_dept_id THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Employee is already in this department';
END IF;

```

```

UPDATE Employee
SET Dept_ID = p_new_dept_id
WHERE Emp_ID = p_emp_id;

```

```

COMMIT;

```

END

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		2	0	0

11. Check Stored Procedure

SHOW PROCEDURE STATUS

WHERE Db = 'db_4546kyfyj';

Db	Name	Type	Language	Definer	Modified	Created	Security_type	Comment	character_set
db_4546kyfyj	ApproveOrRejectLeave	PROCEDURE	SQL	root@%	2026-09-24T05:00:19.000Z	2026-09-24T05:00:19.000Z	DEFINER		utf8mb4
db_4546kyfyj	transfer_employee	PROCEDURE	SQL	user_4546kyfyj@%	2026-09-27T07:29:37.000Z	2026-09-27T07:29:37.000Z	DEFINER		utf8mb4

12. Test the Stored Procedure — Valid Case

First check employee 1:

```

SELECT Emp_ID, Emp_Name, Dept_ID
FROM Employee
WHERE Emp_ID = 1;

```

Emp_ID	Emp_Name	Dept_ID
1	Aarav Sharma	1

Employee 1 initially belongs to department 1.

Now transfer the employee to department 2:

CALL transfer_employee(1, 2);

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		16386	0	0

Verify:

```

SELECT Emp_ID, Emp_Name, Dept_ID
FROM Employee
WHERE Emp_ID = 1;

```

Emp_ID	Emp_Name	Dept_ID
1	Aarav Sharma	2

13. Edge Case — Employee Does Not Exist

CALL transfer_employee(999, 2);

Error: Employee does not exist

14. Edge Case — Employee Already in Department

Employee 1 is now in department 2.

CALL transfer_employee(1, 2);

Error: Employee is already in this department

15. Test Salary Validation — Negative Salary

UPDATE Employee

SET Salary = -5000

WHERE Emp_ID = 2;

Error: Salary must be greater than zero

16. Test Salary Validation — Salary Below Minimum

UPDATE Employee

SET Salary = 5000

WHERE Emp_ID = 2;

Error: Salary cannot be less than 10000

17. Test Salary Validation — Salary Above Maximum

UPDATE Employee

SET Salary = 1500000

WHERE Emp_ID = 2;

Error: Salary cannot exceed 1000000

18. Test Salary Validation — NULL

UPDATE Employee

SET Salary = NULL

WHERE Emp_ID = 2;

Error: Salary cannot be NULL

19. Test Valid Salary Update

UPDATE Employee

SET Salary = 75000

WHERE Emp_ID = 2;

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	1	0	Rows matched: 1 Changed: 1 Warnings: 0	2	0	1

Check:

SELECT Emp_ID, Emp_Name, Salary

FROM Employee

WHERE Emp_ID = 2;

Emp_ID	Emp_Name	Salary
2	Riya Gupta	75000.00

20. Verify Salary Audit

The previous successful salary update should have generated an audit record.

SELECT *

FROM Salary_Audit

ORDER BY Audit_ID DESC;

Audit_ID	Emp_ID	Old_Salary	New_Salary	Changed_At	Action_Type
1	2	62000.00	75000.00	2026-09-27T07:48:23.000Z	SALARY UPDATE

21. Test INSERT Salary Validation

Try inserting an employee with an invalid salary:

INSERT INTO Employee

(Emp_ID, Emp_Name, Salary, Job_Title, Dept_ID, Project_ID)

VALUES

(31, 'Test Employee', -5000, 'Developer', 1, 101);

Error: Salary must be greater than zero

INSERT INTO Employee

(Emp_ID, Emp_Name, Salary, Job_Title, Dept_ID, Project_ID)

VALUES

(31, 'Test Employee', 45000, 'Developer', 1, 101);

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	1	0		2	0	0

22. Final Verification

Check the employee:

SELECT *

FROM Employee

WHERE Emp_ID = 1;

Emp_ID	Emp_Name	Salary	Job_Title	Dept_ID	Project_ID	Manager_ID
1	Aarav Sharma	65000.00	Developer	2	101	null

Check salary audit:

SELECT *

FROM Salary_Audit

ORDER BY Audit_ID;

Audit_ID	Emp_ID	Old_Salary	New_Salary	Changed_At	Action_Type
1	2	62000.00	75000.00	2026-09-27T07:48:23.000Z	SALARY UPDATE

Check the procedure:

SHOW CREATE PROCEDURE transfer_employee;

Procedure	sql_mode	Create P
transfer_employee	IGNORE_SPACE,ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION	CREATE I

Check triggers:

SHOW TRIGGERS;

Trigger	Event	Table	Statement
validate_salary_insert	INSERT	employee	BEGIN IF NEW.Salary IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be NULL'; ELSEIF NEW.Salary <= 0 THEN SIG
validate_salary_update	UPDATE	employee	BEGIN IF NEW.Salary IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be NULL'; ELSEIF NEW.Salary <= 0 THEN SIG
salary_audit	UPDATE	employee	BEGIN IF OLD.Salary <> NEW.Salary THEN INSERT INTO Salary_Audit (Emp_ID, Old_Salary, New_Salary, Action_Type) VALUES (NEW.Emp_ID, OL